In [ ]:
```
#Name : Sharvil Bharatesh Magdum
#PRN : 2425000909
#Roll No : A33
#Batch : A - 2
#Assignment No : 09 [Machine learning -> Random Forest]
```

In [13…
```
#1. Import basic libraries
import numpy as np
import pandas as pd
```

In [6]:
```
#2. Load the dataset 'Iris.xlsx'
data  = pd.DataFrame(pd.read_excel("iris.xlsx"))
data.head(10)
```

Out[6]:

	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width	Class
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
5	5.4	3.9	1.7	0.4	Setosa
6	4.6	3.4	1.4	0.3	Setosa
7	5.0	3.4	1.5	0.2	Setosa
8	4.4	2.9	1.4	0.2	Setosa
9	4.9	3.1	1.5	0.1	Setosa

In [11…
```
#3. Figure out input variables x and output variable y
y = data.iloc[:,4]
x = data.iloc[:,0:4]
print(f"X values:\n{x}")
print(f"Y values:\n{y}")
```

```
X values:
     Sepal_Length  Sepal_Width  Petal_Length  Petal_Width
0             5.1          3.5           1.4          0.2
1             4.9          3.0           1.4          0.2
2             4.7          3.2           1.3          0.2
3             4.6          3.1           1.5          0.2
4             5.0          3.6           1.4          0.2
..            ...          ...           ...          ...
145           6.7          3.0           5.2          2.3
146           6.3          2.5           5.0          1.9
147           6.5          3.0           5.2          2.0
148           6.2          3.4           5.4          2.3
149           5.9          3.0           5.1          1.8

[150 rows x 4 columns]
Y values:
0         Setosa
1         Setosa
2         Setosa
3         Setosa
4         Setosa
        ...
145    Virginica
146    Virginica
147    Virginica
148    Virginica
149    Virginica
Name: Class, Length: 150, dtype: str
```

In [18…
```
#4. Print the shape of x and y.
print(f"X shape:",x.shape)
print(f"Y shape:",y.shape)
```

```
X shape: (150, 4)
Y shape: (150,)
```

In [20…
```
#5. Perform the train test split
from sklearn.model_selection import train_test_split
X_train,X_test,Y_train,Y_test = train_test_split(x,y,test_size = 0.2,rand
```

In [26…
```
#6. Print the shape of the train and test variables
print(f"X_train Shape:{X_train.shape}")
print(f"X_test Shape:{X_test.shape}")
print(f"Y_train Shape:{Y_train.shape}")
print(f"Y_test Shape:{Y_test.shape}")
```

```
X_train Shape:(120, 4)
X_test Shape:(30, 4)
Y_train Shape:(120,)
Y_test Shape:(30,)
```

In [44…
```
#7. Train the model using random forest classifier
from sklearn.ensemble import RandomForestClassifier

RFC = RandomForestClassifier(n_estimators = 100,random_state = 42)
```

```
RFC.fit(X_train,Y_train)

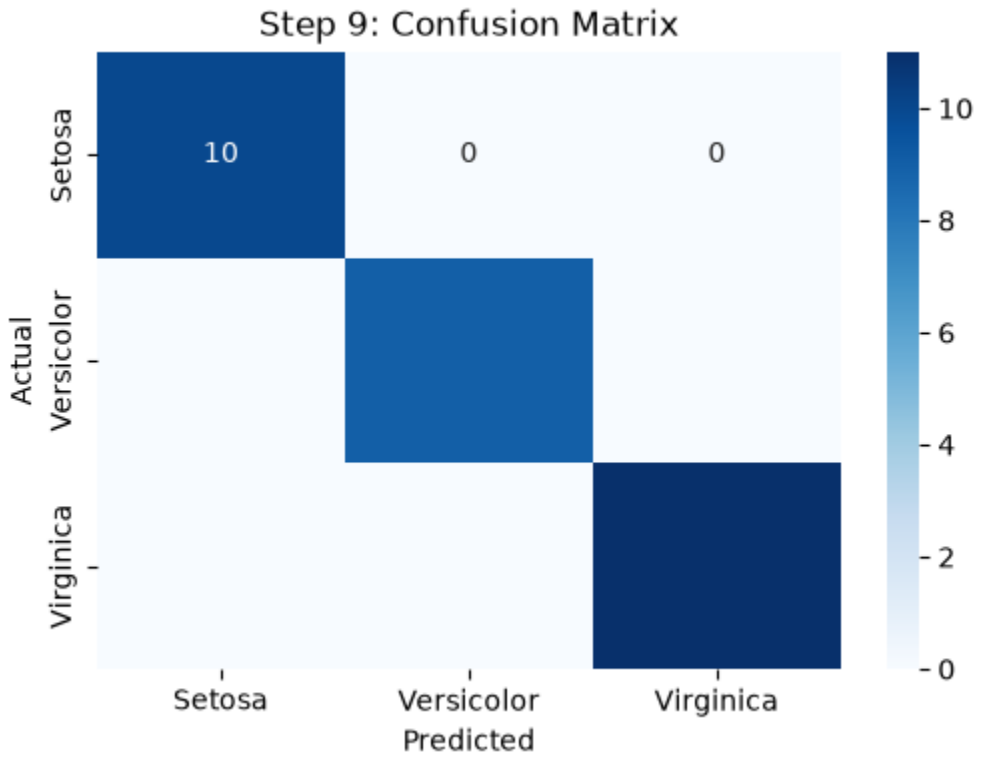
y_pred = RFC.predict(X_test)
```

In [45…
```
#8. Print the Accuracy score
from sklearn.metrics import accuracy_score,classification_report,confusio
acc = accuracy_score(Y_test,y_pred)

print(f"Accuracy Score:{round(acc * 100,2)}")
```

```
Accuracy Score:100.0
```

In [47…
```
#9. Visualize the Confusion Matrix
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(6, 4))
cm = confusion_matrix(Y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels = RFC.classes_,
            yticklabels = RFC.classes_)
plt.title('Step 9: Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



In [48…
```
#10. Print the classification report
classRepo = classification_report(Y_test,y_pred)
print(f"Classification Report:\n{classRepo}")
```

```
Classification Report:
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        10
  Versicolor       1.00      1.00      1.00         9
   Virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

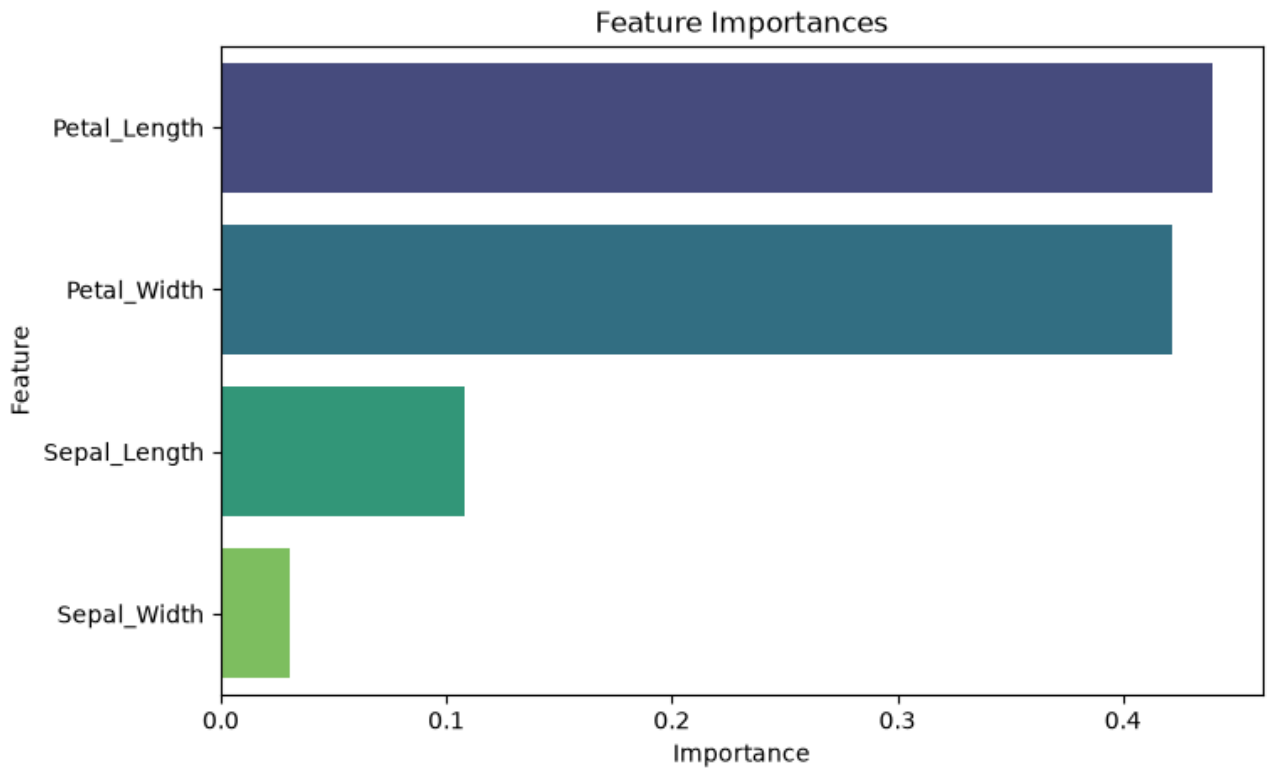
In [54…

```python
#11. Visualize the importance of the parameters and also display their va
importances = RFC.feature_importances_
feature_names = x.columns
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importan
feature_importance_df = feature_importance_df.sort_values(by='Importance'

print("Feature Importances (Descending Order)")
print(feature_importance_df.to_string(index=False))

# Visualizing feature importance
plt.figure(figsize=(8, 5))
sns.barplot(x='Importance', y='Feature', data=feature_importance_df, pale
plt.title('Feature Importances')
plt.show()
```

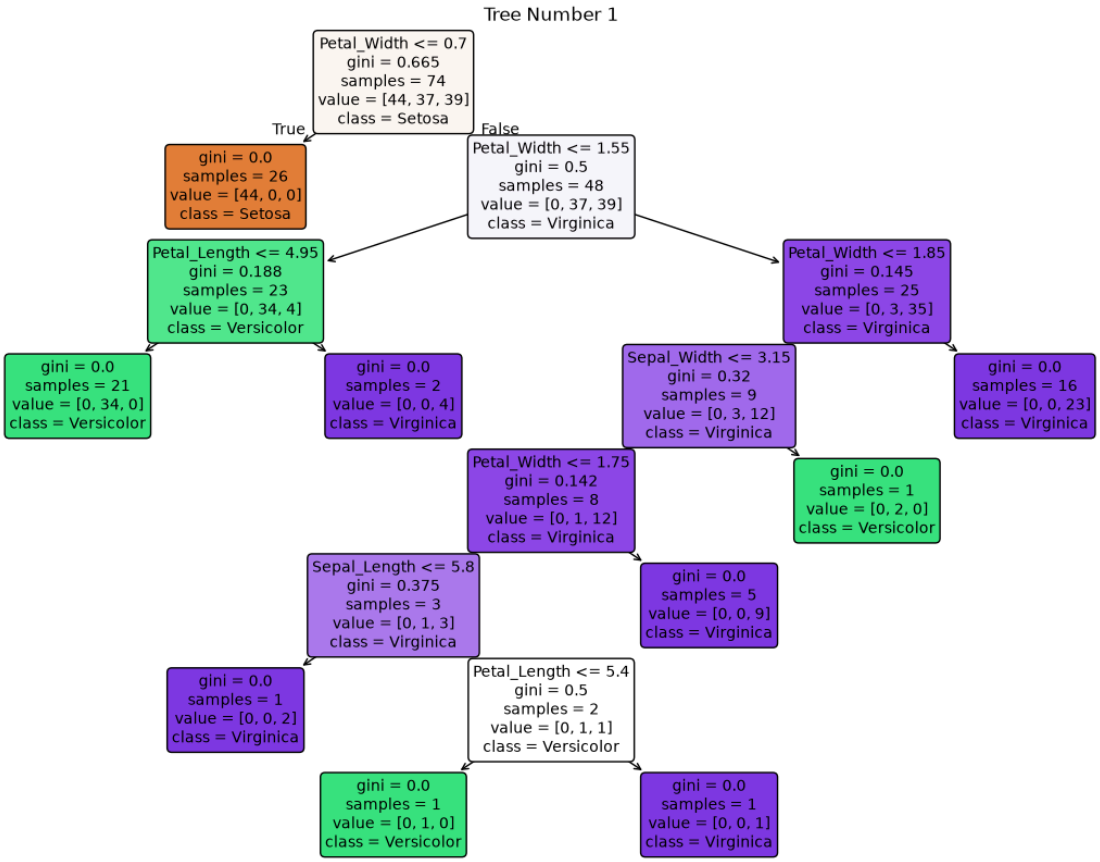
```
Feature Importances (Descending Order)
     Feature  Importance
Petal_Length    0.439994
 Petal_Width    0.421522
Sepal_Length    0.108098
 Sepal_Width    0.030387
```

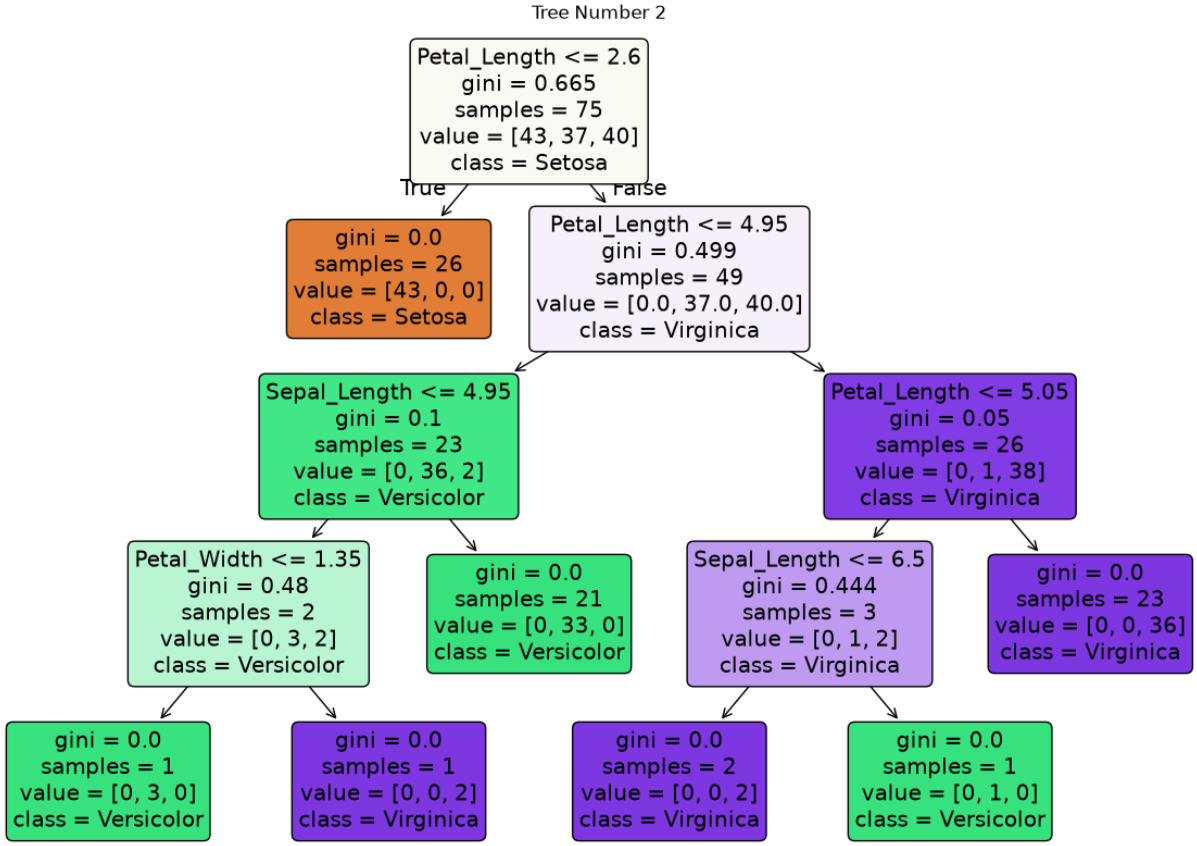


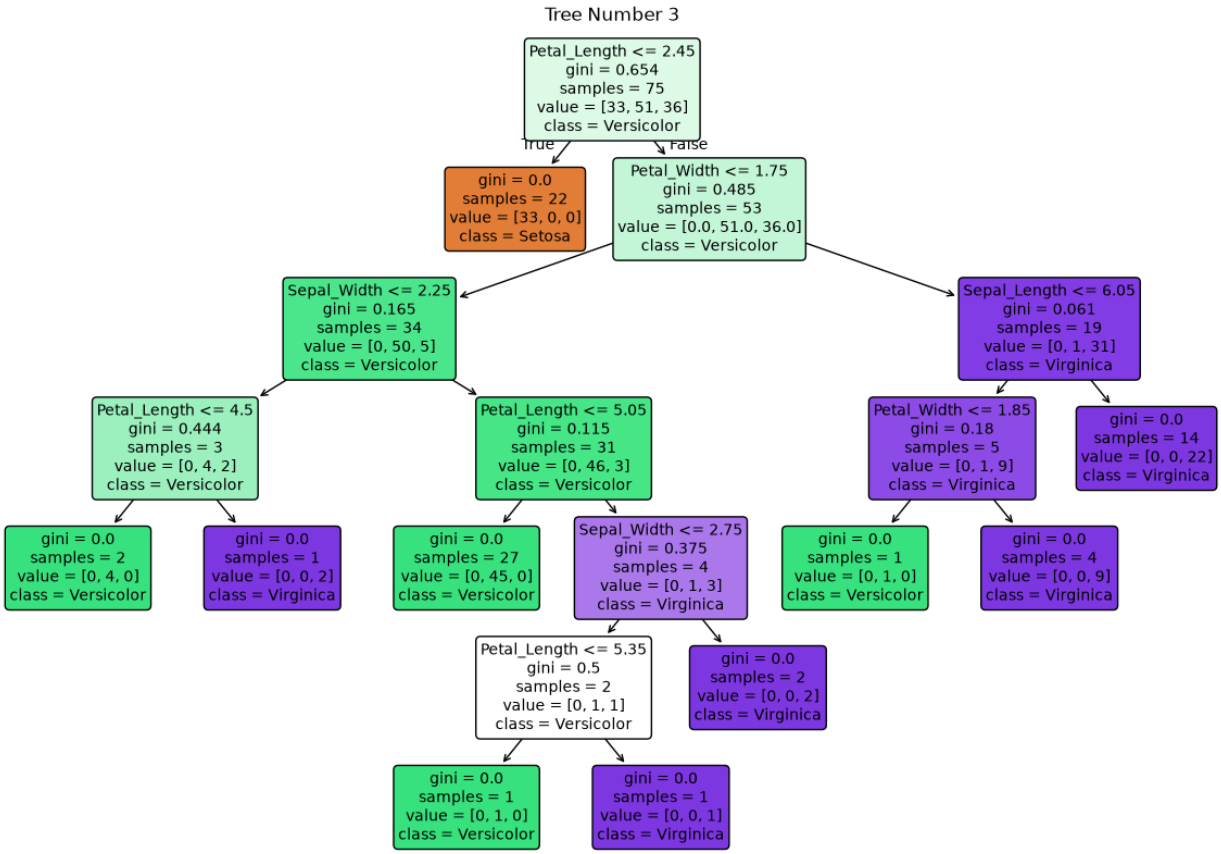
In [57…

```python
#12. Visualize the first n trees
from sklearn import tree
n = 3  # Set how many trees you want to visualize
print(f"Step 12: Visualizing the first {n} trees in the forest")
for i in range(n):
    plt.figure(figsize=(15, 10))
    tree.plot_tree(RFC.estimators_[i],
                   feature_names=feature_names,
                   class_names=RFC.classes_,
                   filled=True,
                   rounded=True)
    plt.title(f'Tree Number {i + 1}')
    plt.show()
```

```
Step 12: Visualizing the first 3 trees in the forest
```


Tree Number 1
Petal_Width <= 0.7
gini = 0.665
samples = 74
value = [44, 37, 39]
class = Setosa
True
False
gini = 0.0
samples = 26
value = [44, 0, 0]
class = Setosa
Petal_Width <= 1.55
gini = 0.5
samples = 48
value = [0, 37, 39]
class = Virginica
Petal_Length <= 4.95
gini = 0.188
samples = 23
value = [0, 34, 4]
class = Versicolor
Petal_Width <= 1.85
gini = 0.145
samples = 25
value = [0, 3, 35]
class = Virginica
gini = 0.0
samples = 21
value = [0, 34, 0]
class = Versicolor
gini = 0.0
samples = 2
value = [0, 0, 4]
class = Virginica
Sepal_Width <= 3.15
gini = 0.32
samples = 9
value = [0, 3, 12]
class = Virginica
gini = 0.0
samples = 16
value = [0, 0, 23]
class = Virginica
Petal_Width <= 1.75
gini = 0.142
samples = 8
value = [0, 1, 12]
class = Virginica
gini = 0.0
samples = 1
value = [0, 2, 0]
class = Versicolor
Sepal_Length <= 5.8
gini = 0.375
samples = 3
value = [0, 1, 3]
class = Virginica
gini = 0.0
samples = 5
value = [0, 0, 9]
class = Virginica
gini = 0.0
samples = 1
value = [0, 0, 2]
class = Virginica
Petal_Length <= 5.4
gini = 0.5
samples = 2
value = [0, 1, 1]
class = Versicolor
gini = 0.0
samples = 1
value = [0, 1, 0]
class = Versicolor
gini = 0.0
samples = 1
value = [0, 0, 1]
class = Virginica


Tree Number 2
Petal_Length <= 2.6
gini = 0.665
samples = 75
value = [43, 37, 40]
class = Setosa
True
False
gini = 0.0
samples = 26
value = [43, 0, 0]
class = Setosa
Petal_Length <= 4.95
gini = 0.499
samples = 49
value = [0.0, 37.0, 40.0]
class = Virginica
Sepal_Length <= 4.95
gini = 0.1
samples = 23
value = [0, 36, 2]
class = Versicolor
Petal_Length <= 5.05
gini = 0.05
samples = 26
value = [0, 1, 38]
class = Virginica
Petal_Width <= 1.35
gini = 0.48
samples = 2
value = [0, 3, 2]
class = Versicolor
gini = 0.0
samples = 21
value = [0, 33, 0]
class = Versicolor
Sepal_Length <= 6.5
gini = 0.444
samples = 3
value = [0, 1, 2]
class = Virginica
gini = 0.0
samples = 23
value = [0, 0, 36]
class = Virginica
gini = 0.0
samples = 1
value = [0, 3, 0]
class = Versicolor
gini = 0.0
samples = 1
value = [0, 0, 2]
class = Virginica
gini = 0.0
samples = 2
value = [0, 0, 2]
class = Virginica
gini = 0.0
samples = 1
value = [0, 1, 0]
class = Versicolor



In [58…

```
#13. Print the total number of trees in the forest
total_trees = len(RFC.estimators_)
print(f"Total trees: {total_trees}")
```

```
Total trees: 100
```